Lab3 运算符重载：String 类

学习目标:

1. 什么是运算符重载以及它如何使程序更具可读性和使编程更方便
2. 运算符重载如何帮助你构造有价值的类
3. 重载一元和二元运算符
4. 将一个类的对象转换成另一个类的对象
5. 何时重载或不重载运算符
6. 创建类 PhoneNumber, Array, String 和 Date 来演示运算符的重载
7. 使用 new 和 delete 完成动态内存分配

实验一: Complex类

1. 问题描述

考虑图10.14-10.16所示的Complex类。该类允许对复数进行运算。复数的形式为：realPart + imaginaryPart * i，其中i的值为 √-1。

a）修改该类，使其通过重载的>>和<<运算符分别输入和输出复数（应该删除该类的print函数）。

b）重载乘法运算符，使两个复数能够执行代数乘法。

c）重载==和!=运算符，支持复数之间的比较。

2. 主函数如下

```
int main()
{
    Complex x, y( 4.3, 8.2 ), z( 3.3, 1.1 ), k;

    cout << "Enter a complex number in the form: (a, b)\n? ";
    cin >> k; // demonstrating overloaded >>
    cout << "x: " << x << "\ny: " << y << "\nz: " << z << "\nk: "
         << k << '\n'; // demonstrating overloaded <<
    x = y + z; // demonstrating overloaded + and =
    cout << "\nx = y + z:\n" << x << " = " << y << " + " << z << '\n';
    x = y - z; // demonstrating overloaded - and =
    cout << "\nx = y - z:\n" << x << " = " << y << " - " << z << '\n';
```

```
    x = y * z; // demonstrating overloaded * and =
    cout << "\nx = y * z:\n" << x << " = " << y << " * " << z << "\n\n";
    if ( x != k ) // demonstrating overloaded !=
        cout << x << " != " << k << '\n';
    cout << '\n';
    x = k;
    if ( x == k ) // demonstrating overloaded ==
        cout << x << " == " << k << '\n';
    return 0;
} // end main
```

3. 输出如下

```
Enter a complex number in the form: (a, b)
? (2, 5)
x: (0, 0)
y: (4.3, 8.2)
z: (3.3, 1.1)
k: (2, 5)

x = y + z:
(7.6, 9.3) = (4.3, 8.2) + (3.3, 1.1)

x = y - z:
(1, 7.1) = (4.3, 8.2) - (3.3, 1.1)

x = y * z:
(5.17, 31.79) = (4.3, 8.2) * (3.3, 1.1)

(5.17, 31.79) != (2, 5)

(2, 5) == (2, 5)
```

实验二: HugeInt类

1. 问题描述

32位整数的机器可以表示的整数范围大约是-20亿~20亿。这一范围的限制一般不会出现问题。不过在有些应用中，我们可能要使用更大范围内的整数。因此，需要创建强大的新数据类型，这也正是C++可以做到的。考虑图10.17－10.19中的HugeInt类。仔细研究该类，然后重载关系运算符、*乘法运算和/除法运算符。

【注意:我们没有给出类HugeInt的赋值运算符或拷贝构造函数，因为编译器提供的赋值运算符和拷贝构造函数能够正确复制整个数组数据成员。】

2. 类定义如下

```
#ifndef HUGEINT_H
```

```
#define HUGEINT_H
#include <iostream>
using std::ostream;
class HugeInt
{
    friend ostream &operator<<( ostream &, const HugeInt & );
public:
    HugeInt( long = 0 ); // conversion/default constructor
    HugeInt( const char * ); // conversion constructor
    // addition operator; HugeInt + HugeInt
    HugeInt operator+( const HugeInt & ) const;
    // addition operator; HugeInt + int
    HugeInt operator+( int ) const;
    // addition operator;
    // HugeInt + string that represents large integer value
    HugeInt operator+( const char * ) const;
    bool operator==( const HugeInt & ) const; // equality operator
    bool operator!=( const HugeInt & ) const; // inequality operator
    bool operator<( const HugeInt & ) const; // less than operator
    // less than or equal to operator
    bool operator<=( const HugeInt & ) const;
    bool operator>( const HugeInt & ) const; // greater than operator

    // greater than or equal to operator
    bool operator>=( const HugeInt & ) const;
    HugeInt operator-( const HugeInt & ) const; // subtraction operator
    HugeInt operator*( const HugeInt & ) const; // multiply two HugeInts
    HugeInt operator/( const HugeInt & ) const; // divide two HugeInts
    int getLength() const;
private:
    short integer[ 30 ];
}; // end class HugeInt
#endif
```

3. 输出如下

```
n1 is 7654321
n2 is 7891234
n3 is 99999999999999999999999999999
n4 is 1
n5 is 12341234
n6 is 7888
result is 0

n1 is not equal to n2
n1 is less than n2
n1 is less than or equal to n2
7654321 + 7891234 = 15545555

99999999999999999999999999999 + 1
= 100000000000000000000000000000

7654321 + 9 = 7654330
7891234 + 10000 = 7901234
12341234 * 7888 = 97347653792
12341234 - 7888 = 12333346
12341234 / 7888 = 1564
```

实验三：字符串连接

1. 问题描述

字符串连接需要两个操作数，即连接的两个字符串。通常，重载的连接运算符会将第二个字符串对象连接到第一个字符串对象的右侧，从而修改第一个字符串对象。但是，在某些应用程序中，希望在不修改字符串参数的情况下生成连接后的字符串对象。本题目要求实现运算符+以允许操作诸如 string 1 = string 2+string 3；其中两个操作数 string 2 和 string 3 都不被修改。

2. 类定义如下

```
#include <iostream>
#include <cstring>
#include <cassert>
using namespace std;

class String
{
    friend ostream &operator<<(ostream &output, const String &s);
public:
    String(const char * const = ""); // conversion constructor
    String(const String &); // copy constructor
    ~String(); // destructor
    const String &operator=(const String &);
    String operator+(const String &);
private:
    char *sPtr; // pointer to start of string
```

```
    int length; // string length
}; // end class String
#endif
```

3. 主函数如下

```
#include <iostream>
#include "String.h"
using namespace std;

int main()
{
    String string1, string2("The date is");
    String string3(" August 1, 1993");

    // test overloaded operators
    cout << "string1 = string2 + string3\n";
    string1 = string2 + string3; // tests overloaded = and + operator
    cout << "\"" << string1 << "\" = \"" << string2 << "\" + \""
        << string3 << "\"" << endl;
    return 0;

}
```

4. 输出如下

```
string1 = string2 + string3
"The date is August 1, 1993" = "The date is" + " August 1, 1993"
```